

Advancing Semi-supervised Anomaly Detection in Software Logs with Deep Grouping and Auto-Optimization

Yicheng Sun^a, Jacky Wai Keung^a, Zhen Yang^{b,*}, Shuo Liu^a and Jingyu Zhang^a

^aDepartment of Computer Science, City University of Hong Kong, Hong Kong, China

^bSchool of Computer Science and Technology, Shandong University, Qingdao, China

ARTICLE INFO

Keywords:

Software Engineering
Log Anomaly Detection
Log Processing
ChatGPT
Sequential Model-based Algorithm Configuration (SMAC)

ABSTRACT

Context: Software logs serve as records generated during the operation of software systems and play a crucial role in identifying anomalies and mitigating system failures. Considering the lack of supervised training examples in practice, semi-supervised approaches have attracted increasing attention in Deep Anomalous Log Detection (DALD) field and achieved promising advancement. Nonetheless, they still face several main challenges. (1) The most widely used rule-based log parsing methods inevitably overlook or misinterpret some dynamic variables owing to diverse log formats, which affects the performance of log grouping during pseudo-label assignment, undermining the capability of DALD tools in follow-up steps. (2) Traditional hyperparameter tuning methods rely on the experience of professionals and trial and error, which is time-consuming and labor-intensive and may not necessarily find the optimal combination of hyperparameters.

Objective: To overcome the aforementioned challenges, we propose SemiSMAC- $\langle T \rangle$, a novel semi-supervised anomaly detection model that leverages ChatGPT for log parsing and grouping, enhancing the efficiency and accuracy of log processing. It can also determine the optimal hyperparameter combinations for different datasets, thus achieving outstanding anomaly detection performance.

Method: In detail, We follow a common workaround to exert ChatGPT for log parsing and then novelly incorporate log grouping into it. The grouping process requires only a few labeled samples for ChatGPT to guide the assignment of pseudo-labels for the rest of the samples, thereby expanding training examples. Additionally, SemiSMAC- $\langle T \rangle$, as a general DALD framework, adopts a random forest-based Bayesian Optimization Algorithm (BOA) to automatically tune the hyperparameters of embedded models. Consequently, efficiently and effectively improving the performance of DALD one-step further, even in low-resource scenarios.

Results: SemiSMAC-LSTM denotes using LSTM as the backbone of SemiSMAC- $\langle T \rangle$, which is the default setting of this framework. Experiments on two popular datasets demonstrate that SemiSMAC-LSTM outperforms six benchmark models, even including three supervised learning models. In the low-resource data setting of 0.1% scale, SemiSMAC-LSTM shows its robustness and resilience, highlighting its effectiveness.

Conclusion: SemiSMAC- $\langle T \rangle$ is capable of handling anomaly detection tasks in both large-scale and low-resource datasets, delivering excellent performance. Therefore, we highly recommend SemiSMAC- $\langle T \rangle$ as an outstanding automated anomaly detection tool.

1. Introduction

Software systems are complex and interconnected, relying on each other to process data effectively. To ensure the high availability and reliability of these interconnected systems, it is crucial to record operational logs [32]. Given the intricate nature of interconnected systems and their large user base, issues are bound to occur. Troubleshooting requires the generation of logs in real-time, often resulting in substantial volumes of log data [56, 53]. Recent studies [84, 16, 3, 77] have focused on anomaly detection methods based on software logs, aiming to identify abnormal behavior by analyzing operational logs and taking timely actions. Deep learning-based algorithms have been proposed to address the reliance of machine learning approaches on manually designed feature representations for log data [48]. These

algorithms can be categorized into unsupervised [8, 81], semi-supervised [41, 79], and supervised [80, 82] learning approaches. Previous anomaly log detection models have demonstrated outstanding performance on public datasets [55, 82, 39].

To ensure the reliability of systems, logs are widely used as they capture key states during system operation [43, 40]. As shown in Figure 1, logs contain dynamic information such as timestamps, numbers, ports, APIs, addresses, block/task IDs, and usernames, which include both static messages and dynamic variables. Currently, many studies have proposed various log parsers [15, 8, 86, 7]. Figure 1 illustrates the log parsing and grouping effects of commonly used log parsers. It can be observed that due to the inability of parsers to fully recognize and replace dynamic variables in logs (e.g., in Logs 4, 5, 7, and 8, some entries omitted usernames and numbers, while some incorrectly parsed addresses.), logs that should have been grouped together (e.g., Logs 6, 7, and 8) are incorrectly divided into two groups. Prior research [24, 86] has shown that this can affect the performance of subsequent anomaly detection models. Therefore, log parsing is a crucial step in

*Corresponding author

✉ yicsun2-c@my.cityu.edu.hk (Y. Sun); Jacky.Keung@cityu.edu.hk (J.W. Keung); zhenyang@sdu.edu.cn (Z. Yang); sliu273-c@my.cityu.edu.hk (S. Liu); jzhang2297-c@my.cityu.edu.hk (J. Zhang)
ORCID(s): 0009-0007-6555-0571 (Y. Sun); 0000-0002-3803-9600 (J.W. Keung); 0000-0003-0670-4538 (Z. Yang); 0000-0002-8877-3678 (S. Liu); 0000-0001-6043-4239 (J. Zhang)

log analysis, and the accuracy of log parsing will impact the performance of downstream tasks [39, 47]. Recently, the rise of Large Language Models (LLMs), like ChatGPT [62], has led to their successful application in various software engineering tasks [76], demonstrating satisfactory performance. Recently, ChatGPT [62] have been successfully applied to various software engineering tasks [76], demonstrating satisfactory performance [12]. Many studies [61, 61, 34] have demonstrated the feasibility of ChatGPT in log parsing. However, using ChatGPT for log grouping has not been explored. We believe that leveraging ChatGPT for both log parsing and grouping offers high fault tolerance and can address the issue of incorrect grouping as depicted in Figure 1, thereby enhancing the accuracy of pseudo-label assignment. Hence, utilizing them for log parsing and grouping presents a new possibility, but further research is needed to ascertain the capabilities of LLMs in this domain.

Group ID (G _{own})	Log ID	Original Log Sequences
1	1	081110 020313 6034 INFO dfc.DataNodePacketResponder: Received block blk_530644381473737599 of size 67108864 from /10.251.195.52
1	2	081110 103655 9020 INFO dfc.DataNodePacketResponder: Received block blk_4935575595350235765 of size 67108864 from /10.251.123.195
2	3	081110 022226 6281 INFO dfc.DataNodeDataXceiver: 10.250.19.16:50010 Served block blk_107800656626961731 to /10.250.19.16
3	4	Jun 15 04:06:18 combo out/pam_unix[21416]: session opened for user cynus by (uid=0)
4	5	- 1121383222 2005.07.14 R01-M0-N0-LJ18-U11 2005-07-14-16:20:22.672549 R01-M0-N0-LJ18-U11 RAS APP FATAL cid: Error creating node map from file /home/pakin/awsep3d-2.2b/results/random1-8x32x32x2.map: Permission denied
5	6	1706/09 20:10:48 INFO python.PythonRunner: Times: total = 41, boot = 15, init = 26, finish = 0
5	7	1706/09 20:10:48 INFO python.PythonRunner: Times: total = 1072, boot = 856, init = 210, finish = 6
5	8	1706/09 20:10:48 INFO python.PythonRunner: Times: total = 1078, boot = 851, init = 219, finish = 6

↓ Log Parsing

Group ID (G _{own})	Group ID (G _{own})	Log ID	Parsed Log Templates
1	1	1	Received block blk_<*> of size <*> from <*>
1	1	2	Received block blk_<*> of size <*> from <*>
2	2	3	<*> Served block blk_<*> to <*>
3	3	4	session opened for user <*> by (uid=0)
4	4	5	cid: Error creating node map from file /home/pakin/awsep3d-2.2b/results/random-8x32x32x2.map: Permission denied
5	5	6	Times: total = <*>, boot = <*>, init = <*>, finish = <*>
6	5	7	Times: total = <*>, boot = <*>, init = <*>, finish = 6
6	5	8	Times: total = <*>, boot = <*>, init = <*>, finish = 6

Figure 1: The example of log parsing and validating the result.

Note: Dynamic variables are highlighted in different colors: red for numbers, grey for ports, green for addresses, purple for block/task IDs, and orange for usernames. We can observe that some dynamic variables are either unresolved or incorrectly resolved.

Typically, supervised methods generally outperform others due to the vast amount of labeled data (e.g., normal or abnormal) used during training. However, these methods are not ideal for industrial applications because they require extensive labeled datasets and are highly sensitive to incorrect labels [32]. This procedure often consumes significant human resources and may lead to labeling errors, which can degrade performance [79]. This makes the effective deployment of unsupervised methods in industry quite difficult. In contrast, semi-supervised methods require only a small amount of labeled data for training, significantly reducing the cost of manual labeling and dependency on labeled data. Unlike unsupervised methods, semi-supervised approaches utilize some training label information (e.g., feature-label correlations), enabling them to optimize the training objective more effectively [46]. Moreover, deep learning models typically have many hyperparameters, such as learning rate,

hidden layer size, number of iterations, and so on. The combination of these hyperparameters can form a huge search space, making manual adjustment very difficult [83]. Traditional methods rely on the experience of professionals and repeated trial and error to adjust hyperparameters. This approach is time-consuming and labor-intensive, and may not necessarily find the optimal combination of hyperparameters.

To address the aforementioned challenges, we introduce a fine-designed ChatGPT-assisted log grouping method for semi-supervised learning and a random forest-based BOA algorithm, namely SMAC, to tune backbones' hyperparameters, thereby advancing the performance of semi-supervised DALD models. We dub this framework as SemiSMAC- $\langle T \rangle$, where $\langle T \rangle$ represents our framework can work with different backbones like genericity in programming. As a semi-supervised learning framework, SemiSMAC- $\langle T \rangle$ only needs a tiny portion of labeled data for initiation. It first adopts ChatGPT to parse log messages to corresponding templates (as shown in Figure 1) and then incorporates log grouping into the above process. As such, even if some of the logs cannot be exactly correctly parsed, ChatGPT can group them to their correct clusters based on semantic understanding, thereby improving the error tolerance of log parsing. Besides, parsed logs that are grouped into the same clusters tend to carry similar semantics. Thus, we can conjecture and assign pseudo-labels for those unlabeled logs based on the existence of labeled logs in their groups, thereby expanding supervised examples for DALD. Afterward, we adopt random forest-based Bayesian Optimization Algorithms (BOA) to automatically tune hyperparameters of the backbone model in SemiSMAC- $\langle T \rangle$, thereby enabling the determination of discrete parameters within deep learning models while requiring less time.

In experiments, we use LSTM, a typical sequential deep neural network, as the default backbone of SemiSMAC- $\langle T \rangle$, namely SemiSMAC-LSTM. Extensive assessments on two popular datasets demonstrate that SemiSMAC-LSTM outperforms six benchmark models, including three supervised learning models, with average improvements of 7.1% and 6.9% in terms of F1-score, respectively. Even in the low-resource data setting of 0.1% scale, SemiSMAC-LSTM shows its robustness and resilience, achieving average increases of 18.7% and 31.2% in F1-scores across the two datasets compared to three representative models, highlighting its effectiveness.

In summary, The contributions of this paper can be summarized in three-fold:

- We innovatively exert ChatGPT to unify log parsing and grouping in semi-supervised DALD, effectively improving the error tolerance of log parsing and the accuracy of pseudo-label assignment.
- We are the first to introduce SMAC method to automatically tune the hyperparameters of backbone models in the DALD field and seamlessly incorporate it into our framework. Various backbones and baselines

are examined in this work, demonstrating its effectiveness in the DALD field.

- We conducted comprehensive experiments to validate the performance of SemiSMAC- $\langle T \rangle$, including comparative assessment, ablation study, and low-resource scenario evaluation.

The remainder of this paper is organized as follows. Section 2 presents the background of anomaly detection. Section 3 elaborates on the proposed approach, including model initialization, log parsing using ChatGPT, pseudo-label generation, log representation, and anomaly detection model building. Sections 4 and 5 discuss the experimental design and results. Section 6 addresses the threats to validity. Section 7 reviews related work. Finally, Section 8 summarizes the paper.

This work is an extension of our COMPSAC 2024 paper [69]. Compared to the preliminary version, we have transitioned the model to semi-supervised learning and innovatively used ChatGPT to unify log parsing and grouping. We have redesigned all experiments and utilized public datasets available on the LogHub [85] platform to facilitate better performance comparisons with existing models (RO2). Additionally, we have compared the performance of ChatGPT against other parsers (RQ3) and evaluated the model's performance on low-resource datasets (RQ4).

2. Background

2.1. Log Parsing

Logs constitute semi-structured text data, typically generated through log statements in source code (e.g., logging.info(), printf(), and Console.WriteLine()) [6]. Each log entry records a specific system event and may include fields like timestamp, severity level, and message content. To facilitate subsequent modeling, log parsing is a necessary step [75]. Log parsing is employed to automatically convert each log message into a structured format by identifying an event template (constant part) associated with parameters (dynamic variable parts) [86, 34]. Figure 2 provides examples of raw log messages and the corresponding log events obtained through parsing. For example, in a software log message like "2023-11-17 00:04:09.377 INFO: 34.139.166.52:0 - 'POST /webhook/order HTTP/1.0' 200 OK delete empty directory: /backend/upload/download/orders/49512/49512", the timestamp is "2023-11-17 00:04:09.377", the severity level is "INFO: 34.139.166.52:0 - 'POST/webhook/order HTTP/1.0' 200 OK", and the message content is "delete empty directory: /backend/upload/download/orders/49512/49512". After appropriate log parsing, this log message can be split into the event template "'POST <*>' 200 OK delete empty directory: <*>", and the parameter values [/webhook/order HTTP/1.0, /backend/upload/download/orders/49512/49512]. Here, "<*>" denotes the position of a parameter.

Various log parsing techniques exist, based on frequent pattern mining [54, 72], clustering [66, 70], and heuristics



Figure 2: The example of log messages and log events.

Note: Dynamic variables are highlighted in red. Irrelevant details (i.e., timestamps, APIs, and software systems) at the top of the log will be removed directly.

[15, 21]. Heuristic methods, capitalizing on log characteristics, have demonstrated superior performance in terms of accuracy [86] and time efficiency [34] compared to other approaches.

The advent of Large Language Models (LLMs) like ChatGPT [4] has showcased considerable promise across diverse domains, spanning conversational interactions [2], language understanding [11], and text extraction [58]. Cheng et al. [4] conducted end-to-end data analysis using ChatGPT across databases from various fields. In log anomaly detection, LogGPT employed ChatGPT, achieving a maximum F1-score of only 0.625 on the BGL dataset, indicating the need for further exploration into this method's viability. Le et al. [33] utilized ChatGPT as a log parser for extracting log events and parameters, evaluating ChatGPT-3.5's performance in log parsing. However, research on ChatGPT's application in software engineering remains limited, especially regarding a thorough investigation into the effectiveness of log parsing with ChatGPT.

2.2. Log Representation

After log parsing, log data must be represented in various formats suitable for deep learning models. Current deep learning-based anomaly detection models typically convert logs into three primary types [32]: (1) Sequential vectors, which reflect the order of log events in a window. DeepLog [8] indexes each log event and subsequently produces a sequential vector for individual log windows. (2) Quantitative vectors, which capture the occurrence of each log event in a log window. LogAnomaly [52] utilizes both sequential and quantitative vectors for anomaly detection. (3) Semantic vectors, which represent log events as distributed embeddings in a vector space, capturing semantic and contextual relationships between different types of log entries [17]. Each log window is converted into a set of semantic vectors for the detection models. For instance, LogRobust [82] employs a pre-trained FastText model to calculate the semantic vectors of log events.

2.3. Sequential Model-based Algorithm Configuration (SMAC)

Sequential Model-Based Optimization (SMBO) [22] is a powerful technique for optimizing complex, expensive, and

black-box objective functions. It iteratively builds a surrogate model, such as Gaussian Processes or Tree-structured Parzen Estimators, to approximate the objective function and guides the search for optimal solutions by maximizing acquisition functions like Expected Improvement. Bayesian Optimization Algorithm (BOA) [60], a popular variant of SMBO, leverages probabilistic models to capture uncertainty and systematically explore the search space, proving highly effective in hyperparameter tuning for neural networks [26]. Neural networks encompass various hyperparameters, which are frequently determined based on empirical knowledge [1]. Identifying the most appropriate parameters for the model through empirical methods can be challenging. These hyperparameters significantly impact the runtime and prediction accuracy of neural networks, making their optimization crucial.

BOA stands as one of the most well-known distribution estimation methods, amalgamating Bayesian networks with evolutionary algorithms to address nearly decomposable problems [68]. BOA is a global optimization algorithm based on Bayes' theorem, suitable for situations where the objective function is difficult to compute or has a high computational cost [59]. Its core idea is to guide the search process by building a probabilistic objective function model to find the parameter configuration that optimizes the objective function [27]. While this approach is highly effective for determining hyperparameters for deep learning models, it has not been widely explored in the field of DALD.

In BOA, we first assume that the objective function (i.e., points determined by different hyperparameters) follows a prior distribution [10]. Then, at each iteration, based on the current probabilistic model of the objective function, we select a point that is most likely to improve performance for evaluation. After evaluation, we incorporate the new observed result into the model and update the probabilistic model [74]. This process continues until a preset number of iterations is reached or other stopping criteria are met. The advantage of BOA lies in its ability to intelligently select the next evaluation point based on historical observations, thereby finding parameter configurations close to the optimal solution in fewer iterations [13, 65].

Conventional Gaussian process-based BOA is limited to handling continuous or numerical variables in the parameter space [20]. However, in the realm of deep learning, numerous parameters are discrete or exhibit conditional relationships with each other [29]. Within the framework of Sequential Model-based Algorithm Configuration (SMAC) [19], Random Forest regression has been proposed as a versatile surrogate model, serving as an alternative to Gaussian Processes. SMAC replaces the surrogate model of BOA with a random forest, which excels in dealing with discrete inputs. It employs a model-based search strategy using Random Forests as its surrogate model to approximate the optimization of the objective function. SMAC's models are based on random forests, a machine learning technique for regression and classification. Random forests consist of multiple regression trees, which are similar to decision trees

and perform well with categorical input data. The overall prediction is made by computing the mean and variance of the outputs from each tree. These ensemble predictions serve as the output of the surrogate model, providing estimates of performance metrics for various hyperparameter configurations.

SMAC uses an acquisition function to balance exploration and exploitation, selecting configurations that are either expected to perform well or are uncertain and worth exploring. This approach necessitates an iterative process, enabling SMAC to enhance the predictive capability of the surrogate model by continuously acquiring new evaluation data when the initial data is limited. Each iteration helps identify configurations with better performance and explores uncertain yet potentially beneficial configurations, thereby improving overall optimization effectiveness through a balance between exploration and exploitation. The iterative process of SMAC starts with an initial set of configurations and their performance data. The surrogate model is trained on this data to capture the relationship between configurations and performance. SMAC then searches the configuration space guided by the surrogate model, proposing new configurations to evaluate. These evaluations update the surrogate model, refining its accuracy and improving the selection of subsequent configurations. This loop continues until a stopping criterion is met, such as a maximum number of evaluations or a satisfactory performance level. SMAC's ability to handle high-dimensional, mixed discrete and continuous configuration spaces makes it particularly effective for complex optimization tasks.

3. Methodology

In this section, we elaborate on the details of our proposed log anomaly detection model.

3.1. Overview

SemiSMAC- $\langle T \rangle$, as a semi-supervised DALD framework, requires only a few labeled log examples to start with. The whole framework consists of five phases: (1) Semi-supervised sample initialization: We select log sequences with different semantics from the original dataset through DBSCAN clustering, manually parse and label them, and use them as grouping prompts for ChatGPT. (2) Log processing based on ChatGPT: We leverage ChatGPT for automated log parsing and grouping. (3) Pseudo-label generation: Based on the grouping results from ChatGPT and the labels of the labeled logs in each group (normal or abnormal), we generate pseudo-labels for other unlabeled logs within the same group. (4) Log representation: We apply an embedding layer to transform discrete log entries into semantic vectors, enriching the log data representation. (5) Anomaly detection model building: We build SemiSMAC-LSTM to perform anomaly detection tasks. SMAC is a powerful framework for optimizing hyperparameters of neural networks. It replaces the surrogate model of Bayesian Optimization Algorithms (BOA) with a random forest, enabling the determination of discrete parameters within deep learning models while

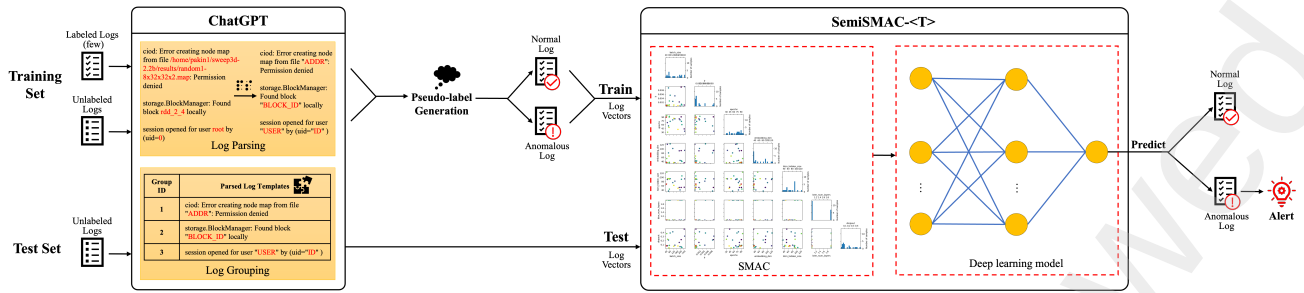


Figure 3: The overview of SemiSMAC-<T>.

requiring less time. With SemiSMAC-<T>, we can automatically tune the hyperparameters to achieve optimal performance, thus overcoming the limitations of manual tuning and static configurations.

3.2. Semi-supervised Sample Initialization

Logs record software anomaly states and are generated in large quantities. For instance, Alibaba’s cloud computing system produces approximately 30-50GB (around 120-200 million lines) of logs per hour [53]. However, after parsing into log templates, many logs are semantically identical. During the preparation phase, we selected a tiny portion of labeled logs, both normal and abnormal, even though some previous studies [8, 52, 79] only labeled normal logs. We believe that many normal and abnormal logs share similar characteristics. By selecting representative logs from both categories, we can avoid abnormal logs being grouped with normal logs by ChatGPT due to similar features, enhancing the grouping accuracy of ChatGPT.

In collaboration with engineers, we utilized the DBSCAN clustering method to cluster the original logs and then manually selected some logs. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [63] is a density-based clustering method that identifies points with high density as cluster cores and assigns other points to their respective clusters based on density connectivity. Its advantages include not requiring a pre-specified number of clusters and effectively identifying noise points [79]. The specific selection steps are as follows: (1) We select 1-2 log sequences from each cluster generated by DBSCAN clustering, and these log sequences are manually confirmed to have the same semantic features. (2) Manually parse the selected log sequences to obtain log templates, label the log templates as normal or abnormal, and then assign a group ID to different templates.

3.3. Log Processing Based on ChatGPT

In this section, we follow previous studies [61, 61, 34] to leverage ChatGPT for log parsing and also innovatively incorporate log grouping that is used for the pseudo-label assignment process into the above process, thereby alleviating the tolerance to error parsing and improving the pseudo-label assignment accuracy. Accordingly, we devised a comprehensive prompt template for ChatGPT to accomplish the above objective, outlined in Figure 4, including (1) Task

Description, (2) Replacement Rules, (3) Grouping Rules, (4) Situated Learning, and (5) Target Log.

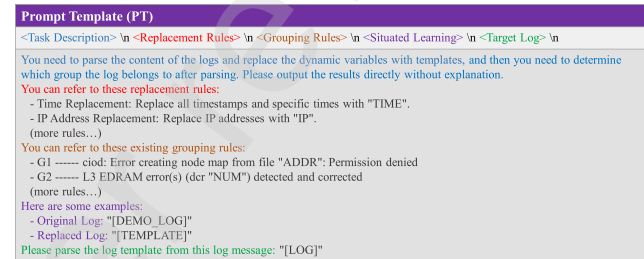


Figure 4: The log parsing prompt template of ChatGPT.

Task Description: This section outlines the log parsing task assigned to ChatGPT, encompassing an introduction, objectives, and anticipated outcomes. Initially, we elucidated the log parsing task to ChatGPT, delineating the expected results. To accomplish this, we employed two prevalent types of ChatGPT instructions: indirect (i.e., Chain-of-Thought prompts) and direct instructions. Indirect instructions included phrases like “replace the dynamic variables” or “parse the content of the logs” to convey our objectives. Direct instructions were more precise, such as “output the result directly, without explanation”.

Replacement Rules: This section elucidates the methodology for extracting and processing crucial information from the log text using ChatGPT. We delineated specific guidelines for replacing or transforming certain text to align with log parsing requirements. For instance, we stipulated rules for substituting file addresses with “ADDR”, IP addresses with “IP”, numerical values (e.g., port numbers) with “NUM”, dates with “TIME”, and usernames with “USER”. This approach aids ChatGPT in comprehending the various types of variables present in the log sequences. We customize the replacement rules for log sequences based on the log formats of different datasets (e.g., the HDFS dataset contains unique block/task IDs). Refining prompts for ChatGPT enables it to effectively parse log formats from different datasets.

Grouping Rules: This section elaborates on enabling ChatGPT to learn grouping information in order to determine log template grouping after parsing. We manually grouped a tiny subset of selected logs and input this grouping

information into ChatGPT, thereby providing it with grouping references.

Situated Learning: Also recognized as in-context learning, this approach underscores the customization of prior knowledge to optimize ChatGPT's practical usefulness and effectiveness in log text processing. Prior studies have indicated that ChatGPT's performance suffers in the absence of prior knowledge. Hence, presenting examples of original and processed log text can enhance ChatGPT's performance in log anomaly detection. This is referred to as "few-shot" prompting, while the absence of such examples constitutes "zero-shot" prompting.

Target Log: This section involves the input of raw log data into ChatGPT. We feed each log text iteratively into the system, and ChatGPT processes them individually, generating output log text data for use in SemiSMAC- $\langle T \rangle$'s anomaly detection experiments. As ChatGPT imposes a maximum length limit on the output tokens for each request, any requests exceeding this limit will result in the termination of the response. Although in our actual experiments, no log text has exceeded the maximum display length, we have implemented additional measures within the program to address potential future log texts that may surpass the maximum length limit. In such instances, ChatGPT will skip processing these log entries and instead provide a log number.

3.4. Pseudo-label Generation

After ChatGPT completes log parsing and grouping, we obtain well-processed log templates and their corresponding groups. At this stage, the log templates represent the form after handling dynamic variables, so log templates within the same group are semantically identical. ChatGPT categorizes log sequences with semantically identical log templates into the same group, indicating that they share a common label. Because we previously selected and grouped a small portion of labeled logs, we can determine which group these parsed log templates might belong to based on whether they contain labeled logs. In other words, if a group contains known log sequences, then new log sequences in that group are more likely to share the same label (i.e., normal or abnormal). Otherwise, these log sequences belong to new groups. Since these groups do not contain labeled logs, pseudo-labels cannot be generated and will be discarded. We assign pseudo-labels to groups containing known log sequences, where the pseudo-labels are determined based on the type of labeled log sequences within that group. Finally, we merge the labeled log samples (initially labeled logs) with the generated pseudo-labeled data (log templates grouped by ChatGPT) to create a new training dataset.

3.5. Log Representation

Log sequences must be converted into numerical vectors before they can serve as input to deep learning models. In our study, we apply an embedding layer to transform discrete log entries into high-dimensional semantic vectors. This technique, inspired by word embeddings in natural language processing (NLP), allows the capture of intricate

contextual relationships within software logs [23]. Each unique identifier in the logs, such as error codes or event types, is associated with a vector that is fine-tuned during the neural network's training process. These semantic vectors enrich the log data representation, facilitating more sophisticated pattern recognition and improving the model's ability to perform downstream tasks like anomaly detection and classification with higher accuracy.

3.6. Anomaly Detection Model Building

The pseudo-labeled dataset constructed will serve as the training set for the anomaly detection framework SemiSMAC- $\langle T \rangle$. We have integrated the SMAC method into the deep learning model to build an effective and efficient anomaly detection model. In this section, we introduce the optimization process using SMAC algorithm. As shown in Figure 5, the application of this software anomaly detection model comprises two stages: 1) Feature selection, model training and optimization. 2) Anomaly detection using well-trained and optimized model. The primary steps are as follows.

3.6.1. Stage 1: Feature selection, model training and optimization

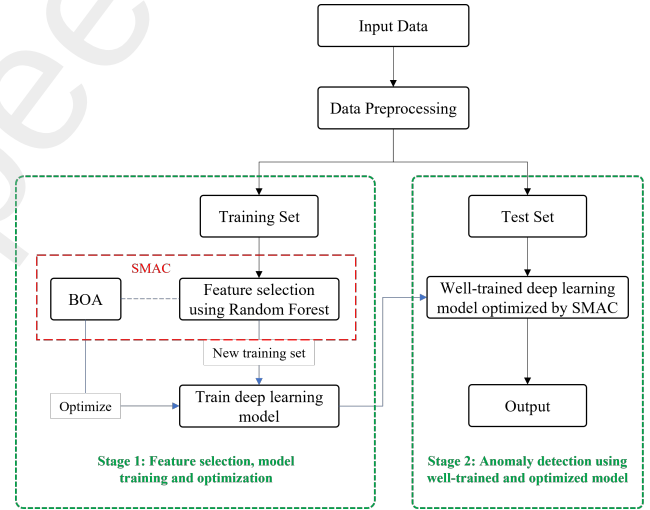


Figure 5: Flowchart of the SemiSMAC- $\langle T \rangle$ model.

Step 1: Construct a training set using initial variables as the input for the deep learning model, then feed this training set into the model. The input of the training set is:

$$Input = [\lambda_{i,j}] = \begin{bmatrix} \lambda_{1,1} & \lambda_{2,1} & \cdots & \lambda_{n,1} \\ \lambda_{1,2} & \lambda_{2,2} & \cdots & \lambda_{n,2} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_{1,m} & \lambda_{2,m} & \cdots & \lambda_{n,m} \end{bmatrix}$$

The output of the training set is:

$$Output = [y_j] = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

where n represents the total number of training samples, and $\lambda_{i,j}$ is the i -th variable of the j -th input sample. $y_j \in \{0, 1\}$ represents the label corresponding to the j -th input sample, with a value of 1 denoting an abnormal label and 0 representing a normal label.

Algorithm 1 Sequential Model-based Algorithm Configuration (SMAC)

- 1: **Initialize:**
 - 2: Randomly sample n_0 points to obtain n_0 results: $\{(x_1, f(x_1)), (x_2, f(x_2)), \dots, (x_{n_0}, f(x_{n_0}))\}$;
 - 3: Determine the maximum number of iterations N .
 - 4: **Model Training:**
 - 5: Train a random forest model using the initial n_0 points.
 - 6: **while** $n \leq N$ **do**
 - 7: Generate candidate points through large-scale random sampling. Use the trained random forest model to calculate the mean and standard deviation of the candidates, and compute the acquisition function $a_n(x)$ for each candidate.
 - 8: Select the point with the highest $a_n(x)$ as x_{n+1} .
 - 9: Evaluate the new point x_{n+1} to get $f(x_{n+1})$. *Note: This step is computationally intensive as it requires model training.*
 - 10: Update the model with $n = n + 1$ points and retrain the random forest.
 - 11: **end while**
 - 12: Return the point with the maximum $f(x)$ among the current evaluated points.
-

Step 2: SMAC constructs the surrogate model of Bayesian Optimization Algorithm (BOA) using a Random Forest, as shown in Algorithm 1. Suppose there are initial sample points $\{(x_1, f(x_1)), (x_2, f(x_2)), \dots, (x_n, f(x_n))\}$ (Line 1-2). Here, x_n represents not sample data points from the dataset but rather hyperparameters of deep learning models, and $f(x_n)$ denotes the anomaly detection performance on the input $([\lambda_{i,j}])$ of the training set under that model's hyperparameter configuration, typically measured by the accuracy or F1-score. Based on these points, we establish a random forest model to fit $f(x_n)$ (Line 4-5). This process is similar to constructing a multidimensional Gaussian distribution formed by n points in Gaussian regression. In SMAC, the primary process of training a random forest as the surrogate model involves: first, sampling with replacement from the initial dataset to build multiple bootstrap sampling subsets; then, training decision trees on each subset by recursively splitting nodes through random feature selection. This process is repeated T times, generating T decision trees. Ultimately, by integrating all decision trees for prediction and uncertainty estimation, an efficient random forest surrogate model approximating the objective function is formed.

Step 3: By training a well-performing random forest and setting the iteration number N in advance (Line 3), SMAC initiates extensive random sampling to determine the optimal candidate points (Line 6-11). Leveraging the capability of random forests to handle discrete variables, we

only need to add conditional restrictions in the parameter space to ensure that certain impossible scenarios are not sampled, or to set ranges or intervals for sampling points, allowing SMAC to conduct sampling. For a new point x , each tree in the random forest provides a prediction, and the average of all tree predictions gives the mean, while the standard deviation of predictions gives the standard deviation. Importantly, the advantage of random forests lies in their prediction complexity for a point x , where each tree's prediction complexity is only $O(\log N)$, significantly reducing prediction time compared to the complex matrix multiplication required in Gaussian regression processes [64].

During the iterative process, SMAC randomly samples multiple points, forming a new multidimensional normal distribution for the newly added point and the previous n points. By determining the marginal distribution of the new point, we obtain its mean and standard deviation. Specifically, for the new point, each tree in the random forest delivers a prediction result; averaging all tree predictions provides the mean, and calculating the standard deviation of predictions yields the standard deviation. We employ Expected Improvement (EI) [73] as the acquisition function $a_n(x)$, selecting x_{n+1} based on the maximum value of $a_n(x)$, and computing $f(x_{n+1})$ for this point. Subsequently, adding this point to the initial set of n points creates a new sample point collection for the next iteration loop.

Step 4: After extensive iterative sampling, the final model will determine an optimal set (i.e. the maximum $f(x)$) of model hyperparameter configurations (Line 12).

3.6.2. Anomaly detection using well-trained and optimized model

Feed the feature variable data from the test set into the well-trained deep learning model and determine the presence of anomaly logs based on the model's output.

After constructing an anomaly detection model using SemiSMAC- $\langle T \rangle$, it can be deployed for real-time system monitoring. When a log sequence is received, SemiSMAC- $\langle T \rangle$ first processes the logs, obtains semantic vectors through log representation, and then uses the well-trained deep learning model to predict whether it is normal or abnormal. If an anomaly is detected, an alert will be promptly issued and sent to engineers for early diagnosis and resolution.

In general, the SMAC method has significant advantages over grid search [36] in hyperparameter optimization. By constructing a probabilistic model for intelligent search, SMAC can find better-performing hyperparameter combinations with fewer evaluations, significantly enhancing efficiency and reducing computational costs. Especially in high-dimensional hyperparameter spaces, SMAC can better balance exploration and exploitation, avoiding the inefficiencies caused by evaluating all possible combinations in grid search.

Table 1

Summary of datasets used in the study.

Dataset	Duration	Numbers	Anomalies
HDFS	38.7 hours	11,175,629 (575,061 blocks)	16,838 blocks (2.93%)
BGL	214.7 days	4,747,963	348,469 (7.34%)

4. Experimental Setup

4.1. Datasets

We employed two public datasets to evaluate the performance of our anomaly detection model: the HDFS dataset [78] and the BGL dataset [57], both available on the LogHub [85] platform. These datasets are labeled with log types and exhibit substantial differences in log formats, enabling us to assess the model's generalization performance. We allocated 80% of the data for training and 20% for testing. The specific distribution of logs is outlined in Table 1.

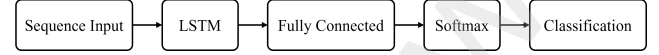
HDFS dataset was generated from over 200 Amazon EC2 nodes, consisting of a total of 11,175,629 log messages. Based on unique block_ids, all log messages were segmented into 575,061 blocks, forming log message sequences, each of which was labeled as normal or abnormal. Among these, 16,838 (2.93%) log blocks are considered to indicate system anomalies.

BGL (Blue Gene/L) dataset is a supercomputing system log dataset collected by Lawrence Livermore National Labs (LLNL). It contains 4,747,963 log messages, each of which is manually labeled as normal or abnormal. Among these, 348,460 (7.34%) log messages are labeled as abnormal.

4.2. Model Parameters

Log data typically exhibits temporal sequences characterized by events with intricate time intervals and patterns. LSTM (Long Short-Term Memory) is particularly suitable for anomaly detection tasks in log data due to its capability to effectively capture and retain long-term dependencies within sequential data. LSTM networks utilize LSTM units as fundamental components in their hidden layers and have proven effective in modeling complex sequences with long-term dependencies [51]. LSTM, equipped with its gated mechanisms including the input gate i_t , forget gate f_t , and output gate o_t , plays a crucial role in managing information flow over time. The forget gate controls the degree to which previous internal state information is retained, while the output gate regulates the flow of information from the current internal state to the external state. By learning and retaining essential features across extended sequences, LSTM can effectively discern anomalous patterns, even when these anomalies are non-contiguous in time or occur at different temporal scales. Consequently, LSTM offers more precise and reliable anomaly detection capabilities compared to traditional approaches relying solely on statistical features or simpler models.

Therefore, we select LSTM as the default anomaly detection model for SemiSMAC- $\langle T \rangle$, and all subsequent experiments are based on SemiSMAC-LSTM. As illustrated in Figure 7, a basic LSTM network architecture is employed, comprising an input layer, LSTM layer, fully connected layer, softmax layer, and output layer.


Figure 6: A simple LSTM network architecture.

The input layer consists of 32 nodes, representing the input for the anomaly log text, which comprises 32 characters. The LSTM layer, also known as the hidden layer, contains multiple LSTM units. Following this is a fully connected layer with 2 output nodes, corresponding to the two classes for prediction. To predict class labels, the network concludes with a softmax layer and classification output layer. The output of the fully connected layer is classified by the softmax function into either 0 or 1. In the softmax layer, classification probabilities are calculated, and the classification output layer categorizes the test data into two groups: events and non-events. In this experiment, the activation function used is the Parametric Rectified Linear Unit (PReLU). PReLU is employed to nonlinearly activate the output features of the convolutional layer. Compared to the classical ReLU function, PReLU enhances model fitting with minimal additional computational cost and a reduced risk of overfitting. The mathematical formula for PReLU is:

$$f_{PReLU(y)} = \begin{cases} y & y > 0 \\ \alpha y & y \leq 0 \end{cases} \quad (1)$$

where y represents the input to a neuron, and α is a small positive parameter learned during training. This allows for a small, positive gradient when the unit is inactive, rather than zero, as in the case of the standard ReLU activation function. PReLU generalizes ReLU by introducing adaptability to the negative part of the function. The challenge of anomaly detection in software logs is typically framed as a binary classification problem. Therefore, A binary cross-entropy loss function is chosen as the cost function, defined as:

$$loss = -[y \log(y^*) + (1 - y) \log(1 - y^*)] \quad (2)$$

Where y represents the actual label, and y^* represents the output of the fully connected layer. The binary cross-entropy loss function quantifies the proximity between the actual output and the expected output.

The weights and biases of the LSTM network are initialized randomly, and the Adaptive Moment Estimation (Adam) algorithm is employed to optimize the internal parameters of the LSTM network during the training process.

Similar to the LSTM, the SemiSMAC-LSTM model incorporates certain parameters present in the LSTM model.

Table 2

The ranges of hyperparameters to be optimized.

Hyperparameters	Value/Range
Batch Size	32, 64, 128, 256
Learning Rate	[0.0001, 0.01]
Epochs	20, 40, 60, 80, 100
Embedding Dimension	32, 64, 128
Hidden Layer Size	32, 64, 128
Number of Layers	1, 2
Dropout Rate	[0, 0.5]

The difference lies in the use of BOA within the SMAC method to determine hyperparameters such as *Batch Size*, *Learning Rate*, *Epochs*, *Embedding Dimension*, *Hidden Layer Size*, *Number of Layers*, and *Dropout Rate*. As shown in Table 2, establishing specific parameter ranges for optimization is crucial to achieve effective optimization results and ensure the feasibility of the optimized hyperparameters.

4.3. Research Questions

This paper mainly focuses on the following five research questions and designs our experiments accordingly.

- **RQ1: What are the optimal hyperparameters of the model under different datasets?**

Neural networks encompass multiple hyperparameters. In traditional LSTM models, hyperparameter settings are often based on empirical knowledge, making it challenging to identify the most suitable parameters through such heuristic methods. This study selects the HDFS and BGL datasets and determines the optimal values for seven hyperparameters of the LSTM model using the SMAC method: *Batch Size*, *Learning Rate*, *Epochs*, *Embedding Dimension*, *Hidden Layer Size*, *Number of Layers*, and *Dropout Rate*. We set the number of iterations to 20. The SMAC method first utilizes BOA to select a combination of hyperparameters, then evaluates the importance of initial variables through a random forest, and records the classification accuracy of the training dataset. The optimal hyperparameter combination is determined through iterative refinement. This method can uncover the optimal parameter combinations for anomaly detection models across different datasets to achieve the best anomaly detection performance.

- **RQ2: What is the performance of SemiSMAC-LSTM against existing DALD approaches?**

In this study, we conducted a comparison between SemiSMAC-LSTM and 6 benchmark models. These benchmark models encompass unsupervised learning-based models (DeepLog), supervised learning-based models (HitAnomaly, NeuralLog, LogRobust), and semi-supervised learning-based models (PLELog, LogBERT). Detailed descriptions of these benchmark

models can be found in Section 4.4. It is noteworthy that the experimental outcomes of DeepLog are sourced from the LogBERT article, while the rest are from their respective original research.

- **RQ3: How does ChatGPT perform in log processing?**

To verify the accuracy of ChatGPT's log processing, we selected all datasets provided on the LogHub platform and compared the results with state-of-the-art parsers. Log parsers such as Drain [15], Logram [8], AEL [86], Spell [7], and SPINE [71] employ various techniques to transform unstructured log data into meaningful information: Drain [15] uses a hierarchical tree for scalability, Logram [8] applies probabilistic models for adaptability, AEL [86] leverages NLP for semantic labeling, Spell [7] groups similar messages through clustering, and SPINE [71] normalizes logs with syntactic pattern recognition. These diverse methods enhance log analysis by addressing different system monitoring challenges. We adopt the version of gpt-4-turbo for log processing and invoke their API¹ provided by OpenAI.

- **RQ4: How does SemiSMAC-LSTM perform on low-resource datasets?**

In practical industrial scenarios, such as factory production lines and navigation robot environments, limited log information is often generated, posing a significant challenge for efficient anomaly detection with minimal data. To systematically evaluate the performance of SemiSMAC-LSTM on low-resource datasets, we selected various scales of the HDFS and BGL datasets, ranging from 0.1% to 15%. Specifically, we experimented with 0.1%, 1%, 5%, 10%, and 15% of the data to verify our approach. Compared to the HDFS dataset, the BGL dataset is larger, with longer log sequences and a more complex structure, making log anomaly detection more challenging. Therefore, the BGL dataset is more suitable for evaluating the model's robustness.

- **RQ5: How generalizable is the SMAC method?**

Traditional deep learning models like Long Short-Term Memory (LSTM) [51], Gated Recurrent Units (GRU) [5], and Convolutional Neural Network (CNN) [38] have demonstrated significant potential in anomaly detection. However, their effectiveness can be limited by their ability to generalize across different datasets and conditions. We aim to further explore the generalization ability of SMAC by determining the optimal hyperparameter combinations for these models to enhance their anomaly detection capabilities.

We combined the SMAC method with traditional deep learning models, namely LSTM, GRU, and RNN, and conducted experiments with six models:

¹<https://platform.openai.com/docs/introduction>

SemiSMAC-LSTM, SemiSMAC-GRU, SemiSMAC-CNN, LSTM, GRU, and CNN on the HDFS and BGL datasets. This evaluation aims to assess the improvement in anomaly detection performance provided by the SMAC method and its generalization effectiveness across different deep learning models. By comparing the performance of these models, we aim to demonstrate the potential of SMAC in enhancing the anomaly detection capabilities of deep learning techniques.

4.4. Benchmark Models

In this section, we present benchmark models for comparison with SemiSMAC-LSTM in terms of performance among various log anomaly detection models. The benchmark models were selected based on the use of a log parser and the learning method.

DeepLog stands as an innovative framework that employs deep learning techniques, notably Long Short-Term Memory (LSTM) networks, to analyze system log data for anomaly detection. It learns the patterns of typical operations from sequential log entries and detects deviations as potential threats or system malfunctions. By emphasizing the sequential aspect of log data, DeepLog accurately predicts and distinguishes between normal and anomalous system behaviors, making it highly effective for security and maintenance tasks in IT systems.

HitAnomaly is a supervised model based on a Transformer architecture. It utilizes a specialized log parser for encoding log information as parameters. Employing a standard log parser, it transforms raw log data into a structured format, which is then analyzed using Transformer encoders. HitAnomaly adopts a classification-based approach, separately encoding both normal and abnormal data types to improve its ability to differentiate between them. This method aims to accurately detect anomalies in system logs by focusing on their unique patterns, making it an invaluable tool for maintaining system integrity and security.

NeuralLog is a supervised classifier model built on the Transformer architecture and incorporates a tokenizer. This strategy empowers the model to learn unique features from raw log data directly without the intermediate step of log parsing, which frequently results in information loss. By integrating a mixture of normal data from the target system and abnormal data from another system, NeuralLog enhances its anomaly detection capabilities, rendering it versatile and efficient for real-world scenarios.

LogRobust is a supervised model built on an attention-based Bi-LSTM architecture. It employs a specialized log parser to preprocess logs, generating TF-IDF and word semantic vectors to extract log features. Both normal and abnormal logs contribute to the training process. The incorporation of attention mechanisms bolsters the model's capacity to pinpoint significant anomalies in log data, thereby enhancing detection accuracy and reliability across various operational contexts.

PLELog introduces a semi-supervised log-based anomaly detection technique that employs probabilistic label estimation to efficiently handle unlabeled log data. This innovative approach integrates semantic information from log events into fixed-length vectors, employs HDBSCAN for clustering these vectors, and utilizes an attention-based GRU network to refine the detection process. Empirical evaluations conducted on datasets such as HDFS and BGL showcase PLELog's robust anomaly detection capabilities, thereby diminishing the necessity for extensive manual labeling.

LogBERT is a BERT-based model for anomaly detection, employing MLM and DeepSVDD losses during training. It processes log sequences refined by a Drain parser to learn normal log patterns through tasks such as masked log key prediction and hypersphere minimization. This strategy enables LogBERT to grasp profound contextual relationships within log data, establishing a robust framework for anomaly detection based on deviations from learned log patterns.

4.5. Evaluation Metrics

4.5.1. Evaluation Metrics for Log Parsing

Following prior studies [6, 24, 30, 42, 86], we use two metrics to evaluate the effectiveness of LLMs in log parsing: Group Accuracy (GA) and Parsing Accuracy (PA).

Group Accuracy (GA) [86] does not directly evaluate the correctness of the parsed logs. Instead, GA assesses the accuracy of the automatically grouped logs (e.g., G_{parsed} shown in Figure 1). GA is calculated as the ratio between the number of correctly grouped logs and the total number of logs. Specifically, GA first groups logs based on the parsed logs (i.e., generated by a log parser), and then compares the resulting groups with those from G_{truth} . For instance, the log parsing result in Figure 1 has a GA of 5/8. Once the raw logs are parsed, they are grouped into different groups. We can see that Logs 1 and 2 are grouped together, Logs 7 and 8 are grouped together, and Logs 3, 4, 5, and 6 form a separate group by themselves. The G_{parsed} for groups 1 to 4 match the grouping results obtained by G_{truth} . However, Logs 6 to 8 form two groups (G_{parsed} 4 and 5) when using the parsed log template, whereas there is only one group (G_{truth} 5) if using the log template from the ground truth. As a result, the GA for this example is 5/8.

Prior studies [6, 24, 42] have highlighted the limitations of GA. For instance, even if the logs are perfectly grouped with a GA of 100%, the parsed log template may not fully match the ground truth template due to unresolved or incorrectly resolved dynamic variables in the parsed templates. As a result, GA cannot indicate whether the logs are parsed correctly. Furthermore, if Logs 4 and 5 are not parsed correctly but are still grouped in a cluster, GA will still be 100.

Parsing Accuracy (PA) [42] requires the log template to match the ground truth template exactly. For example, the log parsing result in Figure 1 has a PA of 1/2 (lower than the GA of 5/8) because there are unresolved or incorrectly resolved dynamic variables in Logs 4, 5, 7, and 8. Hence,

PA is a stricter metric than GA and aligns more closely with practical requirements [24, 30, 42]. Prior studies also found that parsing the dynamic variables can help downstream log analysis tasks [25, 44, 67], further demonstrating the importance of PA over GA.

4.5.2. Evaluation Metrics for Log anomaly detection

Log anomaly detection is treated as a classification problem, and to assess the effectiveness of models in detecting anomalies, we rely on Precision, Recall, and F1-Score metrics to evaluate model performance. The accuracy metric, which calculates the proportion of all predictions correctly made by the anomaly detection model out of the total number of samples, is not used. This is because, in most datasets, normal logs make up the majority, while anomalous logs constitute only a small portion, which is characteristic of well-qualified software. Consequently, a model could attain a high accuracy score even if it predicts all logs as normal. The definitions of each metric are as follows:

Precision can be regarded as a measure of quality, particularly in binary classification tasks. It quantifies the proportion of correctly identified anomalous log sequences out of all the anomalous log sequences detected by the model. A high precision score signifies that the model is reliable and avoids triggering unnecessary alerts.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

Recall evaluates the model's capability to identify all instances of the positive class, representing the percentage of log sequences correctly identified as anomalies out of all anomalies. Recall primarily assesses whether the model can capture all positive samples, and a model with a high recall value indicates that it does not overlook many exceptional cases.

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

The F1-Score represents the harmonic mean of precision and recall. It serves as a metric for assessing the overall effectiveness of a model, particularly when dealing with datasets characterized by imbalanced positive and negative classes. The term "F1" reflects the equal weighting given to precision and recall, making it a commonly used single criterion for evaluating models in the context of anomaly detection.

$$F1-Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5)$$

Specifically, TP (True Positive) is the count of anomalous log sequences correctly detected by the model. FP (False Positive) is the count of normal log sequences incorrectly identified as anomalies. TN (True Negative) is the count of normal logs that are correctly classified. FN (False Negative) is the count of anomalous log sequences that the

Table 3

An Ideal Combination of Hyperparameters Optimized by SMAC under the HDFS and BGL dataset.

Hyperparameters	Values under the HDFS dataset	Values under the BGL dataset
Batch Size	32	256
Learning Rate	0.00919	0.00192
Epochs	37	91
Embedding Dimension	43	92
Hidden Layer Size	126	52
Number of Layers	1	1
Dropout Rate	0.29178	0.22292

model failed to detect. The above four states (TP, FP, TN and FN) can be more effectively visualized and represented using a confusion matrix.

In this paper, we use the F1-Score as the primary metric for evaluating the models. To ensure robustness and mitigate randomness, we conducted each experiment five times and reported the average results. All experiments were performed on a system running Windows 11 with an Intel(R) Core(TM) i7-12700 CPU, 64GB RAM, and an NVIDIA RTX A4000 GPU.

5. Results and Analysis

5.1. Determination of Model Hyperparameters (RQ1)

The initial step in the SemiSMAC-LSTM model involves determining the optimal hyperparameter values for the LSTM model using the SMAC method. A total of 20 iterations were set for this process. The combination of hyperparameters is initially chosen by BOA, and the random forest is used to assess the significance of initial variables. The classification accuracy of the training dataset is recorded, and the optimal set of hyperparameters is determined through iterative steps. Ultimately, an ideal combination of hyperparameters is achieved for the HDFS dataset at the 16th iteration; for the BGL dataset, an ideal combination of hyperparameters is achieved at the 12th iteration, as displayed in Table 3. It is worth noting that BOA operates swiftly as it establishes an alternative hyperparameter search space model and conducts its search within this dimension, rather than the original actual hyperparameter space. In conclusion, the most ideal combination of hyperparameters is preserved, representing the LSTM model parameters derived through the SMAC method.

5.2. Effectiveness of SemiSMAC-LSTM (RQ2)

In this study, we conducted a comparison between SemiSMAC-LSTM and 6 benchmark models, as displayed in Table 4. To summarize, SemiSMAC-LSTM consistently outperforms current state-of-the-art ALD approaches across diverse datasets, highlighting its impressive generality towards heterogeneous log formats and superior anomaly detection abilities.

Table 4

The performance of different models on two datasets.

Model	HDFS			BGL		
	P	R	F1	P	R	F1
Unsupervised learning-based						
DeepLog	0.884	0.695	0.773	0.897	0.828	0.861
Supervised learning-based						
HitAnomaly	1	0.970	0.980	0.950	0.900	0.920
NeuralLog	0.960	1	0.980	0.980	0.980	0.980
LogRobust	0.980	0.960	0.970	0.910	0.770	0.830
Semi-supervised learning-based						
PLELog	0.950	0.963	0.957	0.965	0.999	0.982
LogBERT	0.870	0.781	0.823	0.894	0.923	0.908
SemiSMAC-LSTM	0.981	0.989	0.985	0.980	0.989	0.983

As can be seen from Table 4, All semi-supervised and fully supervised models outperformed the unsupervised model DeepLog, highlighting the crucial importance of leveraging historical samples in anomaly log detection tasks. Owing to the log format discrepancy between the above two datasets, leading to the log parsers of each model can hardly be generalized to them concurrently. Thus, most ALD models cannot perform consistently well on both datasets above. HitAnomaly and LogRobust demonstrate notably higher performance on the HDFS dataset compared to their performance on the BGL dataset. Conversely, Deeplog, PLELog, and LogBERT exhibit the opposite pattern. In contrast, SemiSMAC-LSTM demonstrates stable information extraction capabilities in logs of different formats. Notably, SemiSMAC-LSTM, as a semi-supervised approach, even outperforms all three supervised learning models, namely LogRobust, NeuralLog, and HitAnomaly, by 1.5%, 0.5%, and 0.5% on the HDFS dataset, respectively. It also outperforms them by 15.3%, 0.3%, and 6.3% on the BGL dataset in order, showcasing a more competitive advantage.

5.3. Analysis on ChatGPT as the Log Parser (RQ3)

In this section, we evaluate ChatGPT's performance in log parsing and grouping. Experiments were conducted on a benchmark dataset provided by the LogHub² [85] platform. This dataset contains logs from 16 open-source systems and is widely used to evaluate and compare the accuracy of log parsers [6, 7, 15]. The studied systems span various domains, including distributed systems, operating systems, mobile systems, supercomputers, server applications, and standalone software. Each system includes 2,000 log messages along with their respective log templates and parameters, serving as the ground truth for evaluating log parsers. It is worth noting that the dataset in Section 4.1 is sourced from the HDFS and BGL datasets provided by the LogHub platform. We assess the performance of each log parser

²<https://github.com/logpai/loghub>

through Grouping Accuracy (GA) and Parsing Accuracy (PA). Table 5 shows both the grouping accuracy (GA) and parsing accuracy (PA) of the state-of-the-art methods and our designed ChatGPT Parser.

Overall, due to the design of specific prompts for ChatGPT's *Replacement Rules*, *Grouping Rules*, and *Situated Learning* sections tailored to each dataset, its performance is significantly superior. Specifically, for the GA metric, our method achieved the best results on 12 out of 16 datasets, with an average score 9.8% higher than the widely used rule-based parser, Drain. For the PA metric, the evaluation standard is more stringent: log sequences are considered failed if even a single dynamic variable is not successfully identified. This results in existing parsers averaging a high score of 0.861 in GA but only 0.335 in PA. With our prompt-specific approach, ChatGPT excels at distinguishing dynamic variables in log messages, successfully extracting and processing them, ultimately achieving an average PA score of 0.947, a 67.8% improvement over other parsers. Figure 7 illustrates some log templates generated by our method and state-of-the-art methods after log parsing.

Original Log Sequence	Our ChatGPT Parser	State-of-the-art Parsers
generating core.385	generating core:"CORE_ID" generating core:"NUM"	generating core.<*>
cioid: Error creating node map from file /home/pakin1/sweep3d-2.2b/results/random- 8x32x32x2.map: Permission denied	cioid: Error creating node map from file "ADDR": Permission denied	cioid: Error creating node map from file /home/pakin1/sweep3d-2.2b/results/random- <*>: Permission denied
Received block blk_-967145856473901804 of size 67108864 from /10.250.6.191	Received block blk_"NUM" of size "NUM" from "IP"	Received block blk.<*> of size <*> from <*>
storage.BlockManager: Found block rdd_2_4 locally	storage.BlockManager: Found block "BLOCK_ID" locally	storage.BlockManager: Found block rdd_2_4 locally
cioid: Error loading /g/g24/buber/Yunisc/BlueGene/partad.devel/t adriver.32.exe: program image too big. 361544528 > 266076160	cioid: Error loading "ADDR": program image too big. "NUM" > "NUM"	cioid: Error loading /g/g24/buber/Yunisc/BlueGene/partad.devel/t adriver.<*>.exe: program image too big. <*> > <*>
session opened for user root by (uid=0)	session opened for user "USER" by (uid="ID")	session opened for user root by (uid=0)

Figure 7: Examples of log templates after log parsing.

From Figure 7, we can see that rule-based log parsers struggle to identify many specific dynamic log variables (such as domain URLs, APIs, addresses, block/task IDs, usernames, etc.) due to significant variations in their occurrences in log data. This leads to lower PA scores. In contrast, ChatGPT effectively identifies and processes these variables after learning from our few-shot prompts. For instance, ChatGPT correctly identified address variables in the second and fifth log messages, as well as block IDs, usernames, and UIDs in the fourth and sixth log messages, which were not recognized by the rule-based parser. However, during the processing stage, ChatGPT can also be affected by noise. For example, in the first log message, it may occasionally replace numbers with "ID" or "CORE_ID" instead of accurately with "NUM". Even with more detailed prompts, this issue cannot be entirely avoided.

5.4. Exploration in Low-resource Scenarios (RQ4)

As mentioned in Section 4.3, there are currently numerous small-scale datasets where traditional anomaly detection models may not perform exceptionally well due to the limited data volume. To validate the performance of SemiSMAC-LSTM on low-resource datasets, we selected

Table 5

A comparison of the grouping accuracy (GA) and parsing accuracy (PA) for the state-of-the-art methods and our designed ChatGPT parser (the last column).

	Drain		Logram		AEL		Spell		SPINE		<i>Our Parser</i>	
	GA	PA	GA	PA	GA	PA	GA	PA	GA	PA	GA	PA
Distributed systems												
HDFS	0.998	0.355	0.930	0.005	0.998	0.625	1	0.301	0.866	0.499	1	0.974
Hadoop	0.948	0.269	0.451	0.113	0.869	0.262	0.778	0.113	0.950	0.279	0.982	0.984
Spark	0.920	0.360	0.282	0.259	0.905	0.360	0.905	0.321	0.925	0.337	0.998	0.991
Zookeeper	0.967	0.497	0.724	0.474	0.921	0.496	0.964	0.452	0.989	0.502	1	0.993
OpenStack	0.733	0.019	0.326	0	0.758	0.019	0.764	0	0.384	0.011	1	0.926
Super computers												
BGL	0.963	0.342	0.587	0.125	0.957	0.344	0.787	0.197	0.923	0.376	0.961	0.978
HPC	0.887	0.636	0.911	0.643	0.903	0.678	0.654	0.530	0.945	0.667	0.975	0.980
Thunderbird	0.955	0.047	0.554	0.004	0.941	0.036	0.844	0.027	0.665	0.051	0.950	0.912
Operating systems												
Windows	0.997	0.462	0.694	0.141	0.690	0.153	0.989	0.004	0.684	0.151	0.993	0.979
Linux	0.690	0.184	0.361	0.124	0.405	0.174	0.152	0.088	0.545	0.108	0.898	0.935
Mac	0.787	0.218	0.568	0.169	0.764	0.169	0.757	0.033	0.761	0.204	0.785	0.705
Mobile systems												
Android	0.831	0.548	0.742	0.278	0.773	0.393	0.863	0.150	0.938	0.181	0.885	0.913
HealthApp	0.780	0.231	0.267	0.112	0.568	0.163	0.639	0.152	0.983	0.446	0.905	0.885
Server applications												
Apache	1	0.694	0.313	0.007	1	0.694	1	0.694	1	0.276	1	1
OpenSSH	0.789	0.508	0.611	0.298	0.537	0.246	0.556	0.191	0.676	0.253	1	1
Standalone software												
Proxifier	0.527	0	0.504	0	0.495	0.495	0.527	0.478	0.049	0.016	1	1
Average	0.861	0.335	0.551	0.172	0.780	0.331	0.761	0.233	0.767	0.272	0.959	0.947

Note: The highest values for GA and PA for each system are highlighted in **bold**. The results for Drain and Logram are based on the evaluation conducted by Khan et al. [24] on the corrected log dataset, while the results for the others are from the research by Le et al. [25].

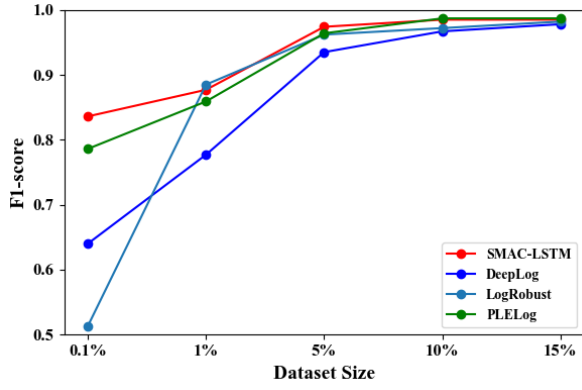
various proportions of the HDFS and BGL datasets, creating different sizes of low-resource datasets, specifically 0.1%, 1%, 5%, 10%, and 15%. For comparison, we chose one model from each category: unsupervised learning-based models (DeepLog), supervised learning-based models (LogRobust), and semi-supervised learning-based models (PLELog). Figure 8 illustrates the F1-scores of each model on the two datasets. The reference data for DeepLog, PLELog, and LogRobust experiments is derived from Le's [31] research.

As can be seen from Figure 8, with the reduction of data size to the scale of 15%-0.1%, The performance of DeepLog and PLELog decreases by an average of 19.7%-28.4% and 3.5%-20.5% in terms of F1-score. As for the fully supervised approach, namely LogRobust, its also performance decreases by 21.6% at most. For SemiSMAC-LSTM, the anomaly detection performance slightly decreases in low-resource datasets, but the reduction is extremely limited. Even when the data scale is reduced to 0.1%, the F1-score on the HDFS and BGL datasets only decreases by 14.9% and 17.8%, respectively. Furthermore, as the data scale reaches 10%, its F1-score stabilizes, remaining equivalent

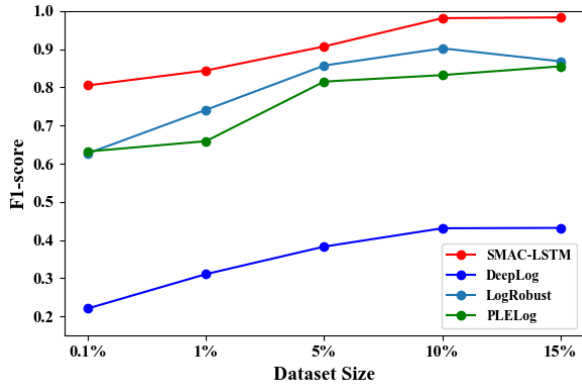
to that at 100% scale, indicating the highly robust nature of SemiSMAC-LSTM in anomaly detection.

In comparison, the other three models exhibit unstable anomaly detection performance in low-resource datasets, especially at a data scale of 0.1%. For instance, as shown in Figure 8a, with the HDFS dataset at a 0.1% data scale, SemiSMAC-LSTM's F1-score improves by 19.6%, 5.0%, and 17.8% compared to DeepLog, PLELog, and LogRobust, respectively. As the dataset scale increases, all models experience significant improvements in F1-scores when the scale reaches 1%, with respective increases of 15.8% (DeepLog), 10.55% (PLELog), 7.7% (LogRobust), and 4.1% (SemiSMAC-LSTM). Once the dataset scale reaches 10%, all models achieve stable F1-scores.

As shown in Figure 8b, with the BGL dataset at a 0.1% data scale, SemiSMAC-LSTM achieves an F1-score of 0.805, surpassing PLELog and LogRobust by 17.3% and 17.8%, respectively. Even LogRobust, trained under full supervision, struggles to learn log sequence features in extremely limited data conditions without the most suitable



(a) HDFS



(b) BGL

Figure 8: The performance of models on low-resource datasets.

hyperparameter combinations for the attention-based Bi-LSTM architecture. As the dataset size increases, the F1-scores of all four models improve, but LogRobust shows some fluctuations. At a dataset size of 15%, LogRobust's F1-score is 3.4%, which is lower than that at the data size of 10%. Additionally, at a dataset size of 15%, SemiSMAC-LSTM achieves an F1-score of 0.983, marking a 26.5% average increase over the other three models.

It is worth noting that due to the variation in dataset scales, the hyperparameters determined by SemiSMAC-LSTM also change accordingly. Table 6 presents the optimal hyperparameter configurations optimized by SMAC at the 0.1% scale of the HDFS and BGL datasets. A comparison between Table 6 and Table 3 reveals that, for different scales and types of datasets, SemiSMAC-LSTM can consistently determine the optimal hyperparameter combination through iterative processes. Furthermore, SemiSMAC-LSTM's performance in anomaly detection with the determined optimal hyperparameters outperforms other models, demonstrating its robustness in handling imbalanced and low-resource datasets. This also implies that SemiSMAC-LSTM's capability in identifying anomalous logs is reliable.

Table 6

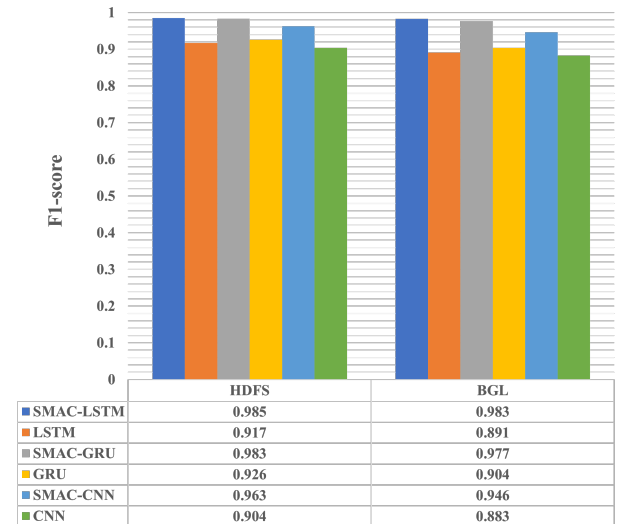
The optimal hyperparameter configuration for SMAC at the 0.1% scale of the HDFS and BGL datasets.

Hyperparameters	Values under the HDFS dataset	Values under the BGL dataset
Batch Size	32	128
Learning Rate	0.00686	0.00993
Epochs	63	24
Embedding Dimension	39	52
Hidden Layer Size	78	126
Number of Layers	1	1
Dropout Rate	0.45466	0.31586

Overall, SemiSMAC-LSTM consistently outperforms other models of different learning paradigms and demonstrates robustness in diverse data scales. Besides, its consistent and superior performance on both datasets also highlights its generality capabilities.

5.5. Exploration of Generalization in SMAC (RQ5)

Figure 9 illustrates the F1-scores of six models, including SemiSMAC-LSTM, SemiSMAC-GRU, SemiSMAC-CNN, LSTM, GRU, and CNN, on the HDFS and BGL datasets. In general, after incorporating the SMAC method, the F1-scores of the three deep learning models improved by an average of 6.13% and 7.6% on the two datasets. This indicates the effectiveness of the SMAC method and its remarkable generalization ability when combined with different deep learning models, thereby enhancing their performance in anomaly detection.


Figure 9: The performance of SMAC method across different deep learning models.

Furthermore, when comparing the models individually, both LSTM and GRU achieved higher F1-scores than the CNN model. Additionally, with the inclusion of the SMAC

method, LSTM demonstrated the most significant improvement, with F1-scores increasing by 6.8% and 9.2% on the two datasets, respectively. It is worth noting that all models exhibited greater improvements on the BGL dataset compared to the HDFS dataset.

6. Threats to Validity

This section clarifies the threats to log formatting, noise in ChatGPT, and system upgrades, respectively.

6.1. Log Formatting

Logs are produced during software runtime via output statements authored by programmers. Consequently, the definition and output format of logs may differ among programmers across various countries and regions. In certain instances, logs might even be generated using the programmer's native language. Hence, to accommodate different formats and encompass all dynamic variables, it is imperative to establish distinct 'extraction and replacement rules' tailored to log formats for ChatGPT. This will enhance ChatGPT's accuracy in comprehending logs.

6.2. Noise in ChatGPT

In Section 5.3, we investigated the performance of ChatGPT in log parsing. As indicated by the results, ChatGPT inevitably introduces noise during processing. Consequently, a few logs may fail to undergo replacement for certain dynamic variables. The introduction of such noise presents a challenge to the subsequent model's anomaly detection predictions. Currently, we are exploring a wider range of prompt templates to mitigate or eliminate the potential for noise.

6.3. System Upgrades

In real-world production settings, log formats may change due to system upgrades, even within the same system. When a large influx of new logs occurs, fine-tuning might be needed to ensure the model remains effective. Additionally, as technology progresses and requirements evolve, the emergence of new log formats is expected, and existing formats may also undergo enhancements and extensions. Our future efforts will involve analyzing log data from diverse datasets to better cater to the needs of anomaly detection tasks.

7. Related Work

7.1. ChatGPT for Log Processing

The advent of Large Language Models (LLMs) like ChatGPT [4] has showcased considerable promise across diverse domains, spanning conversational interactions [2], language understanding [11], and text extraction [58]. Cheng et al. [4] conducted end-to-end data analysis using ChatGPT-4.0 across databases from various fields. In log anomaly detection, LogGPT [61] employed ChatGPT, achieving a maximum F1-score of only 0.618 on the BGL dataset, indicating the need for further exploration into this method's viability. Le et al. [34] utilized ChatGPT as a log parser for

extracting log events and parameters, evaluating ChatGPT-3.5's performance in log parsing. However, research on ChatGPT's application in software engineering remains limited, especially regarding a thorough investigation into the effectiveness of log parsing with ChatGPT. It remains unclear whether ChatGPT can effectively conduct automatic log parsing and how its performance compares to that of other log parsers.

7.2. Log Anomaly Detection Models

To ensure system reliability, logs are well suited for tasks because they record key states during system operation. Currently, a substantial number of data-driven methods have been introduced that automatically detect anomalies in systems by analyzing software logs. The challenge of anomaly detection in software logs is commonly regarded as a binary classification problem [28]. Currently, researchers have proposed various methods for detecting abnormal logs from large-scale log datasets. Traditional machine learning techniques can mine hidden information from various data sources. Lou et al. [43] were the pioneers in applying Invariant Mining (IM) to log anomaly detection, utilizing an algorithm to extract sparse integer invariants from event count vectors. These extracted invariants can unveil inherent linear characteristics within the program's workflow, and log sequences that deviate from these invariant principles are considered anomalous. He et al. [40] conducted an evaluation of the performance of six machine learning-based anomaly log detection algorithms; however, these algorithms exhibited issues such as low efficiency and limited flexibility. The emergence of deep learning algorithms has addressed these challenges. Limitations encountered in machine learning for log data anomaly detection include the requirement for substantial amounts of labeled data to train models effectively, the challenge of generalizing across various system behaviors, susceptibility to adversarial attacks simulating normal patterns, and complexities in deciphering intricate model decisions. Furthermore, evolving systems can render acquired patterns obsolete, thus demanding ongoing retraining and adaptation.

To address these issues and tackle the common reliance of machine learning-based approaches on manually designed feature representations for log data, deep learning-based algorithms for log anomaly detection have been introduced. LogRobust [82] and CNN [45] are notable deep learning algorithms in the supervised learning category, while DeepLog [8], LogAnomaly [52], Logsy [55], and Autoencoder [9] are prominent unsupervised learning algorithms. Furthermore, Zhang et al. [82] introduced LogRobust, which represents log events as semantic vectors and uses an attention-based Bi-LSTM model to detect anomalies. Huang et al. [18] were the first to apply the transformer model to the task of log anomaly detection; they proposed HitAnomaly, which used a hierarchical transformer to model sequences of log templates and parameter values and designed an attention mechanism to merge semantic

information with parameter values for classification, demonstrating robustness to volatile log data. Although supervised methods provide accurate diagnoses of system conditions, their main limitation is the need for a substantial volume of normal and abnormal data during the training phase. Therefore, researchers have also explored unsupervised and semi-supervised approaches [37, 50]. Du et al. [8] employed DeepLog to treat log information as natural language sequences, autonomously learn log patterns from normal execution sequences, and identify anomalies by assessing whether incoming log events deviate from the predictions of a stacked LSTM model. Meng et al. [52] devised a template representation method, termed template2vec. This method involves matching incoming log events with existing templates rather than directly flagging them as anomalies, thereby enhancing accuracy. NeuralLog [31] introduces a novel log-based anomaly detection method based on a Transformer classification model that eliminates the need for log preprocessing. Yang et al. [41, 79] introduced a semi-supervised learning framework known as PLELog, which utilizes the HDBSCAN [49] clustering method for estimating labels probabilistically on unlabeled log sequences, addressing the challenge of limited labeling. LogBERT [14] learns patterns within normal log sequences through masked log message prediction and hypersphere volume minimization, yet it cannot specifically identify semantic information in abnormal logs. LAnoBERT [35] employs unsupervised learning using the BERT model to detect anomalous logs, although it does not fully consider interactive factors between normal and abnormal logs.

8. Conclusion

The SemiSMAC- $\langle T \rangle$ model introduced in this research paper has showcased its tremendous potential in the domain of software log anomaly detection. We select SemiSMAC-LSTM as the default model for SemiSMAC- $\langle T \rangle$. By leveraging the strengths of LSTM in capturing temporal dependencies and incorporating context-dependent mechanisms through a Bayesian optimization algorithm based on random forests, SemiSMAC-LSTM has demonstrated enhanced sensitivity to subtle anomalies within time series data that are often overlooked by traditional LSTM models. We explored the use of the SMAC method, which has been effective in optimizing hyperparameters and improving model performance in various log anomaly detection tasks. The SMAC method's capability to adaptively search for the best configuration makes it a powerful tool for enhancing deep learning models.

Moving forward, our future research endeavors will focus on exploring the scalability of SemiSMAC- $\langle T \rangle$ in handling larger datasets and assessing its applicability to a wider range of anomaly detection tasks. Additionally, investigating the interpretability of self-attention mechanisms within the model can provide deeper insights into its decision-making processes, thereby enhancing its transparency and trustworthiness.

It is worth highlighting that SemiSMAC- $\langle T \rangle$ possesses the remarkable ability to adaptively focus on key features without incurring significant computational complexity. This attribute positions it as a valuable tool in the field of anomaly detection. Moreover, considering its significant potential, SemiSMAC- $\langle T \rangle$ can find application in diverse domains such as network security, financial fraud detection, and healthcare monitoring systems. We anticipate that the utilization of SemiSMAC- $\langle T \rangle$ will expand in the future, contributing to improved anomaly detection and proactive risk mitigation in various industries.

Data availability: We release our source code publicly at <https://github.com/log-detection/SemiSMAC-LSTM>

Acknowledgment

This work is supported in part by the General Research Fund of the Research Grants Council of Hong Kong and the research funds of the City University of Hong Kong (6000796, 9229109, 9229098, 9220103, 9229029).

References

- [1] Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B., 2011. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems* 24.
- [2] Chen, N., Wang, Y., Jiang, H., Cai, D., Chen, Z., Li, J., 2022. What would harry say? building dialogue agents for characters in a story. *arXiv preprint arXiv:2211.06869*.
- [3] Chen, Z., Liu, J., Gu, W., Su, Y., Lyu, M.R., 2021. Experience report: Deep learning-based system log analysis for anomaly detection. *arXiv preprint arXiv:2107.05908*.
- [4] Cheng, L., Li, X., Bing, L., 2023. Is gpt-4 a good data analyst? *arXiv preprint arXiv:2305.15038*.
- [5] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- [6] Dai, H., Li, H., Chen, C.S., Shang, W., Chen, T.H., 2020. Logram: Efficient log parsing using n -gram dictionaries. *IEEE Transactions on Software Engineering* 48, 879–892.
- [7] Du, M., Li, F., 2018. Spell: Online streaming parsing of large unstructured system logs. *IEEE Transactions on Knowledge and Data Engineering* 31, 2213–2227.
- [8] Du, M., Li, F., Zheng, G., Srikumar, V., 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning, in: *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pp. 1285–1298.
- [9] Farzad, A., Gulliver, T.A., 2020. Unsupervised log message anomaly detection. *ICT Express* 6, 229–237.
- [10] Frazier, P.I., 2018. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*.
- [11] Frieder, S., Pinchetti, L., Griffiths, R.R., Salvatori, T., Lukasiewicz, T., Petersen, P., Berner, J., 2024. Mathematical capabilities of chatgpt. *Advances in Neural Information Processing Systems* 36.
- [12] Gao, S., Wen, X.C., Gao, C., Wang, W., Zhang, H., Lyu, M.R., 2023. What makes good in-context demonstrations for code intelligence tasks with llms?, in: *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE. pp. 761–773.
- [13] Garnett, R., 2023. *Bayesian optimization*. Cambridge University Press.
- [14] Guo, H., Yuan, S., Wu, X., 2021. Logbert: Log anomaly detection via bert, in: *2021 international joint conference on neural networks (IJCNN)*, IEEE. pp. 1–8.

- [15] He, P., Zhu, J., Zheng, Z., Lyu, M.R., 2017. Drain: An online log parsing approach with fixed depth tree, in: 2017 IEEE international conference on web services (ICWS), IEEE. pp. 33–40.
- [16] He, S., Zhu, J., He, P., Lyu, M.R., 2016. Experience report: System log analysis for anomaly detection, in: 2016 IEEE 27th international symposium on software reliability engineering (ISSRE), IEEE. pp. 207–218.
- [17] Hu, C., Sun, X., Dai, H., Zhang, H., Liu, H., 2023. Research on log anomaly detection based on sentence-bert. *Electronics* 12, 3580.
- [18] Huang, S., Liu, Y., Fung, C., He, R., Zhao, Y., Yang, H., Luan, Z., 2020. Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE transactions on network and service management* 17, 2064–2076.
- [19] Hutter, F., Hoos, H.H., Leyton-Brown, K., 2010. Sequential model-based optimization for general algorithm configuration (extended version). Technical Report TR-2010-10, University of British Columbia, Computer Science, Tech. Rep. .
- [20] Hutter, F., Hoos, H.H., Leyton-Brown, K., 2011. Sequential model-based optimization for general algorithm configuration, in: *Learning and Intelligent Optimization: 5th International Conference, LION 5*, Rome, Italy, January 17–21, 2011. Selected Papers 5, Springer. pp. 507–523.
- [21] Jiang, Z.M., Hassan, A.E., Flora, P., Hamann, G., 2008. Abstracting execution logs to execution events for enterprise applications (short paper), in: 2008 The Eighth International Conference on Quality Software, IEEE. pp. 181–186.
- [22] Jones, D.R., Schonlau, M., Welch, W.J., 1998. Efficient global optimization of expensive black-box functions. *Journal of Global optimization* 13, 455–492.
- [23] Joulin, A., Grave, E., Bojanowski, P., Douze, M., Jégou, H., Mikolov, T., 2016. Fasttext. zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651* .
- [24] Khan, Z.A., Shin, D., Bianculli, D., Briand, L., 2022. Guidelines for assessing the accuracy of log message template identification techniques, in: *Proceedings of the 44th International Conference on Software Engineering*, pp. 1095–1106.
- [25] Khan, Z.A., Shin, D., Bianculli, D., Briand, L., 2023. Impact of log parsing on log-based anomaly detection. *arXiv preprint arXiv:2305.15897* .
- [26] Klein, A., Falkner, S., Bartels, S., Hennig, P., Hutter, F., 2017. Fast bayesian hyperparameter optimization on large datasets .
- [27] Kobliha, M., Schwarz, J., Očenášek, J., 2006. Bayesian optimization algorithms for dynamic problems, in: *Applications of Evolutionary Computing: EvoWorkshops 2006: EvoBIO, EvoCOMNET, EvoHOT, EvoASP, EvoINTERACTION, EvoMUSART, and EvoSTOC*, Budapest, Hungary, April 10–12, 2006. Proceedings, Springer. pp. 800–804.
- [28] Landauer, M., Onder, S., Skopik, F., Wurzenberger, M., 2023. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications* 12, 100470.
- [29] Laumanns, M., Ocenasek, J., 2002. Bayesian optimization algorithms for multi-objective optimization, in: *International Conference on Parallel Problem Solving from Nature*, Springer. pp. 298–307.
- [30] Le, V., Zhang, H., . Log parsing with prompt-based few-shot learning. *arxiv* 2023. *arXiv preprint arXiv:2302.07435* .
- [31] Le, V.H., Zhang, H., 2021. Log-based anomaly detection without log parsing, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE. pp. 492–504.
- [32] Le, V.H., Zhang, H., 2022a. Log-based anomaly detection with deep learning: How far are we?, in: *Proceedings of the 44th international conference on software engineering*, pp. 1356–1367.
- [33] Le, V.H., Zhang, H., 2022b. Log-based anomaly detection with deep learning: How far are we?, in: *Proceedings of the 44th international conference on software engineering*, pp. 1356–1367.
- [34] Le, V.H., Zhang, H., 2023. Log parsing with prompt-based few-shot learning. *arXiv preprint arXiv:2302.07435* .
- [35] Lee, Y., Kim, J., Kang, P., 2023. Lanobert: System log anomaly detection based on bert masked language model. *Applied Soft Computing* 146, 110689.
- [36] Lerman, P., 1980. Fitting segmented regression models by grid search. *Journal of the Royal Statistical Society Series C: Applied Statistics* 29, 77–84.
- [37] Li, X., Chen, P., Jing, L., He, Z., Yu, G., 2020. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults, in: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), IEEE. pp. 92–103.
- [38] Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J., 2021. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems* 33, 6999–7019.
- [39] Li, Z., Luo, C., Chen, T.H., Shang, W., He, S., Lin, Q., Zhang, D., 2023. Did we miss something important? studying and exploring variable-aware log abstraction, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 830–842.
- [40] Liang, Y., Zhang, Y., Xiong, H., Sahoo, R., 2007. Failure prediction in ibm bluegene/l event logs, in: *Seventh IEEE International Conference on Data Mining (ICDM 2007)*, IEEE. pp. 583–588.
- [41] Lin, Y., Chiang, Y.Y., 2022. A semi-supervised learning approach for abnormal event prediction on large network operation time-series data, in: 2022 IEEE International Conference on Big Data (Big Data), IEEE. pp. 1024–1033.
- [42] Liu, Y., Zhang, X., He, S., Zhang, H., Li, L., Kang, Y., Xu, Y., Ma, M., Lin, Q., Dang, Y., et al., 2022. Uniparser: A unified log parser for heterogeneous log data, in: *Proceedings of the ACM Web Conference 2022*, pp. 1893–1901.
- [43] Lou, J.G., Fu, Q., Yang, S., Xu, Y., Li, J., 2010. Mining invariants from console logs for system problem detection, in: 2010 USENIX Annual Technical Conference (USENIX ATC 10).
- [44] Lu, S., Rao, B., Wei, X., Tak, B., Wang, L., Wang, L., 2017. Log-based abnormal task detection and root cause analysis for spark, in: 2017 IEEE International Conference on Web Services (ICWS), IEEE. pp. 389–396.
- [45] Lu, S., Wei, X., Li, Y., Wang, L., 2018. Detecting anomaly in big data system logs using convolutional neural network, in: 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), IEEE. pp. 151–158.
- [46] Ma, X., Keung, J., He, P., Xiao, Y., Yu, X., Li, Y., 2024a. A semisupervised approach for industrial anomaly detection via self-adaptive clustering. *IEEE Transactions on Industrial Informatics* 20, 1687–1697.
- [47] Ma, Z., Chen, A.R., Kim, D.J., Chen, T.H.P., Wang, S., 2024b. Llmpraser: An exploratory study on using large language models for log parsing .
- [48] Manning, C.D., 2009. An introduction to information retrieval. Cambridge university press.
- [49] McInnes, L., Healy, J., 2017. Accelerated hierarchical density based clustering, in: 2017 IEEE International Conference on Data Mining Workshops (ICDMW), IEEE. pp. 33–42.
- [50] McInnes, L., Healy, J., Astels, S., 2017. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.* 2, 205.
- [51] Memory, L.S.T., 2010. Long short-term memory. *Neural computation* 9, 1735–1780.
- [52] Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., et al., 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs., in: *IJCAI*, pp. 4739–4745.
- [53] Mi, H., Wang, H., Zhou, Y., Lyu, M.R.T., Cai, H., 2013. Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems. *IEEE Transactions on Parallel and Distributed Systems* 24, 1245–1255.
- [54] Nagappan, M., Vouk, M.A., 2010. Abstracting log lines to log event types for mining software system logs, in: 2010 7th IEEE Working

- Conference on Mining Software Repositories (MSR 2010), IEEE. pp. 114–117.
- [55] Nedelkoski, S., Bogatinovski, J., Acker, A., Cardoso, J., Kao, O., 2020. Self-attentive classification-based anomaly detection in unstructured logs, in: 2020 IEEE International Conference on Data Mining (ICDM), IEEE. pp. 1196–1201.
- [56] Nyyssälä, J., Mäntylä, M., Varela, M., 2022. How to configure masked event anomaly detection on software logs?, in: 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE. pp. 414–418.
- [57] Oliner, A., Stearley, J., 2007. What supercomputers say: A study of five system logs, in: 37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07), IEEE. pp. 575–584.
- [58] Olmo, A., Sreedharan, S., Kambhampati, S., 2021. Gpt3-to-plan: Extracting plans from text using gpt-3. arXiv preprint arXiv:2106.07131.
- [59] Pelikan, M., 2002. Bayesian optimization algorithm: From single level to hierarchy. University of Illinois at Urbana-Champaign.
- [60] Pelikan, M., Pelikan, M., 2005. Bayesian optimization algorithm: Hierarchical Bayesian optimization algorithm: toward a new generation of evolutionary algorithms, 31–48.
- [61] Qi, J., Huang, S., Luan, Z., Yang, S., Fung, C., Yang, H., Qian, D., Shang, J., Xiao, Z., Wu, Z., 2023. Loggpt: Exploring chatgpt for log-based anomaly detection, in: 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), IEEE. pp. 273–280.
- [62] Roumeliotis, K.I., Tselikas, N.D., 2023. Chatgpt and open-ai models: A preliminary review. Future Internet 15, 192.
- [63] Schubert, E., Sander, J., Ester, M., Kriegel, H.P., Xu, X., 2017. Dbscan revisited, revisited: why and how you should (still) use dbscan. ACM Transactions on Database Systems (TODS) 42, 1–21.
- [64] Shahriari, B., Swersky, K., Wang, Z., Adams, R.P., De Freitas, N., 2015. Taking the human out of the loop: A review of bayesian optimization. Proceedings of the IEEE 104, 148–175.
- [65] Shang, Q., Tan, D., Gao, S., Feng, L., et al., 2019. A hybrid method for traffic incident duration prediction using boa-optimized random forest combined with neighborhood components analysis. Journal of Advanced Transportation 2019.
- [66] Shima, K., 2016. Length matters: Clustering system log messages using length of words. arXiv preprint arXiv:1611.03213.
- [67] Shin, D., Khan, Z.A., Bianculli, D., Briand, L., 2021. A theoretical framework for understanding the relationship between log parsing and anomaly detection, in: International Conference on Runtime Verification, Springer. pp. 277–287.
- [68] Snoek, J., Larochelle, H., Adams, R.P., 2012. Practical bayesian optimization of machine learning algorithms. Advances in neural information processing systems 25.
- [69] Sun, Y., Keung, J., Zhang, J., Yu, H.K., Luo, W., Liu, S., 2024. Unveiling hidden anomalies: Leveraging smac-lstm for enhanced software log analysis, in: 2024 IEEE 48th International Conference on Computers, Software, and Applications (COMPSAC), IEEE.
- [70] Tang, L., Li, T., Perng, C.S., 2011. Logsig: Generating system events from raw textual logs, in: Proceedings of the 20th ACM international conference on Information and knowledge management, pp. 785–794.
- [71] Vaarandi, R., 2003. A data clustering algorithm for mining patterns from event logs, in: Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM 2003)(IEEE Cat. No. 03EX764), Ieee. pp. 119–126.
- [72] Vaarandi, R., Pihelgas, M., 2015. Logcluster-a data clustering and pattern mining algorithm for event logs, in: 2015 11th International conference on network and service management (CNSM), IEEE. pp. 1–7.
- [73] Vazquez, E., Bect, J., 2010. Convergence properties of the expected improvement algorithm with fixed mean and covariance functions. Journal of Statistical Planning and inference 140, 3088–3095.
- [74] Wang, X., Jin, Y., Schmitt, S., Olhofer, M., 2023. Recent advances in bayesian optimization. ACM Computing Surveys 55, 1–36.
- [75] Wang, X., Zhang, X., Li, L., He, S., Zhang, H., Liu, Y., Zheng, L., Kang, Y., Lin, Q., Dang, Y., et al., 2022. Spine: a scalable log parser with feedback guidance, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1198–1208.
- [76] White, J., Hays, S., Fu, Q., Spencer-Smith, J., Schmidt, D.C., 2024. Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, in: Generative AI for Effective Software Development. Springer, pp. 71–108.
- [77] Wu, X., Li, H., Khomh, F., 2023. On the effectiveness of log representation for log-based anomaly detection. Empirical Software Engineering 28, 137.
- [78] Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I., 2009. Detecting large-scale system problems by mining console logs, in: Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles, pp. 117–132.
- [79] Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., Zhang, W., 2021. Semi-supervised log-based anomaly detection via probabilistic label estimation, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE. pp. 1448–1460.
- [80] Ying, S., Wang, B., Wang, L., Li, Q., Zhao, Y., Shang, J., Huang, H., Cheng, G., Yang, Z., Geng, J., 2021. An improved knn-based efficient log anomaly detection method with automatically labeled samples. ACM Transactions on Knowledge Discovery from Data (TKDD) 15, 1–22.
- [81] Zeufack, V., Kim, D., Seo, D., Lee, A., 2021. An unsupervised anomaly detection framework for detecting anomalies in real time through network system's log files analysis. High-Confidence Computing 1, 100030.
- [82] Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., et al., 2019. Robust log-based anomaly detection on unstable log data, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 807–817.
- [83] Zhao, J., Li, D., Zhou, J., Armaghani, D.J., Zhou, A., 2024. Performance evaluation of rock fragmentation prediction based on rf-boa, adaboost-boa, gboost-boa, and ert-boa hybrid models. Deep Underground Science and Engineering.
- [84] Zhu, J., He, P., Fu, Q., Zhang, H., Lyu, M.R., Zhang, D., 2015. Learning to log: Helping developers make informed logging decisions, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, IEEE. pp. 415–425.
- [85] Zhu, J., He, S., He, P., Liu, J., Lyu, M.R., 2023. Loghub: A large collection of system log datasets for ai-driven log analytics, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), IEEE. pp. 355–366.
- [86] Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., Lyu, M.R., 2019. Tools and benchmarks for automated log parsing, in: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE. pp. 121–130.